LAPORAN

WORKSHOP SISTEM INFORMASI

WEB SERVER

(TIF330805)

SEMESTER III



BKPM Acara 18 JSON encoding dan HTTP status code

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

RESTful API merupakan antarmuka yang digunakan untuk menghubungkan client dengan server melalui protokol HTTP. Dalam pengembangan aplikasi berbasis web, API memerlukan format pertukaran data yang terstruktur agar dapat dipahami oleh client. Salah satu format yang umum digunakan adalah JSON (JavaScript Object Notation).

Selain format data, API juga perlu menggunakan HTTP status code untuk memberikan informasi mengenai hasil pemrosesan request. Status tersebut menunjukkan apakah permintaan berhasil diproses atau mengalami kesalahan. Pada PHP Native, JSON dapat dibentuk menggunakan fungsi `json_encode()`, sedangkan status HTTP dapat ditentukan menggunakan fungsi `http_response_code()`.

Pada praktikum Acara 18, dilakukan pengembangan RESTful API pada Sistem Informasi Akademik dengan membuat class `Response.php` pada lapisan Core. Class tersebut digunakan untuk menyeragamkan format respons JSON dan pengaturan HTTP status code. Pengembangan dilakukan dengan melanjutkan proyek dari Acara 17, sehingga struktur Model, Controller, dan Router yang telah dibuat dapat digunakan kembali.

1.2 Tujuan Praktikum

1. Memahami penerapan JSON encoding pada PHP Native.
2. Memahami penggunaan HTTP status code dalam RESTful API.
3. Membuat class `Response.php` untuk mengelola respons JSON.
4. Menerapkan format respons yang konsisten pada Controller dan Router.
5. Menguji RESTful API menggunakan Postman.

BAB 2. LANDASAN TEORI

2.1 JSON Encoding dan Decoding

JSON (JavaScript Object Notation) merupakan format pertukaran data yang ringan dan mudah dibaca oleh manusia maupun mesin. JSON digunakan untuk mengirimkan data antara client dan server dalam aplikasi berbasis web.

PHP menyediakan fungsi `json_encode()` untuk mengubah array atau objek menjadi teks JSON. Sementara itu, fungsi `json_decode()` digunakan untuk mengubah teks JSON menjadi struktur data PHP agar dapat diproses oleh program.

2.2 HTTP Status Code

HTTP status code merupakan kode numerik yang dikirimkan server untuk menunjukkan hasil pemrosesan request. Penggunaan status code membantu client mengetahui apakah permintaan berhasil diproses atau mengalami kesalahan.

Beberapa HTTP status code yang umum digunakan dalam RESTful API adalah sebagai berikut.

Kode	Keterangan
200 OK	Request berhasil diproses
201 Created	Data baru berhasil dibuat
204 No Content	Request berhasil tanpa isi respons
400 Bad Request	Request tidak valid
401 Unauthorized	Autentikasi tidak valid atau tidak tersedia
403 Forbidden	Akses tidak diizinkan
404 Not Found	Data atau endpoint tidak ditemukan
405 Method Not Allowed	Metode HTTP tidak diizinkan
409 Conflict	Terjadi konflik data
422 Unprocessable Entity	Data tidak memenuhi aturan validasi
500 Internal Server Error	Terjadi kesalahan pada server

2.3 Response Envelope

Response envelope merupakan pola penyusunan respons API dengan struktur yang konsisten. Respons dapat berisi indikator keberhasilan, pesan, data, dan

timestamp. Struktur tersebut memudahkan client dalam membaca dan memproses hasil request.

Contoh struktur respons JSON:

```
{
  "success": true,
  "message": "Data berhasil diambil",
  "data": [],
  "timestamp": "2026-10-25T10:00:00+00:00"
}
```

Nilai pada contoh tersebut hanya ilustrasi struktur respons. Timestamp dan data aktual akan mengikuti hasil pemrosesan aplikasi.

2.4 Class Response

Class Response merupakan class pada lapisan Core yang bertanggung jawab mengatur format respons JSON dan HTTP status code. Dengan menggunakan class ini, proses pembentukan respons tidak perlu ditulis berulang kali pada setiap method Controller.

2.5 Postman

Postman merupakan aplikasi yang digunakan untuk menguji API dengan mengirimkan request HTTP ke server. Postman mendukung metode GET, POST, PUT, DELETE, dan metode lainnya. Hasil pengujian dapat diamati melalui status HTTP, header, dan isi respons JSON.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktikum

3.1.1 Studi Kasus

Studi kasus pada Acara 18 adalah menerapkan JSON Encoding dan HTTP Status Code pada REST API menggunakan PHP Native MVC. Implementasi dilakukan dengan menambahkan class `Response` pada lapisan Core, memperbarui Controller dan Router, serta menguji response berhasil maupun gagal.

A. Langkah 1 — Membuat Class Response pada Lapisan Core

Langkah pertama adalah membuat file `Response.php` pada folder `app/core/`. Class ini digunakan untuk menyeragamkan format response JSON yang dikirimkan API kepada client.

Pada file `app/core/Response.php`, dibuat method `json()` untuk mengatur HTTP status code, tipe konten, struktur response, dan proses encoding data menjadi JSON.

```php
<?php
class Response {

    public static function json(int $httpCode, bool
$success, string $message, $data = null, array $errors =
[]): void {
        http_response_code($httpCode);
        header('Content-Type: application/json;
charset=utf-8');

        $body = [
            'success' => $success,
            'message' => $message,
            'data'    => $data,
        ];

        // Field errors hanya muncul jika ada kesalahan
validasi
        if (!empty($errors)) {
            $body['errors'] = $errors;
        }

        $body['timestamp'] = date('c');
```

```
        $flags = JSON_UNESCAPED_SLASHES |
JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE;

        // Tugas Mandiri 2: PRETTY_PRINT hanya pada mode
pengembangan
        if (self::isDevelopment()) {
            $flags |= JSON_PRETTY_PRINT;
        }

        echo json_encode($body, $flags);
        exit;
    }

    private static function isDevelopment(): bool {
        return defined('APP_ENV') && APP_ENV ===
'development';
    }
}
```

Method `json()` menerima kode HTTP, status keberhasilan, pesan, data, dan daftar kesalahan. Data disusun dalam format yang konsisten sebelum diubah menjadi JSON menggunakan `json_encode()`. Field `errors` hanya ditambahkan apabila terdapat kesalahan, sedangkan `timestamp` digunakan untuk mencatat waktu response dibuat.

B. Langkah 2 — Merefaktor MahasiswaController.php

Langkah kedua adalah memperbarui `MahasiswaController.php` dengan mengganti pemanggilan `http_response_code()` dan `echo json_encode()` menjadi `Response::json()`.

Contoh implementasi pada method `index()` adalah sebagai berikut.

```
<?php

class MahasiswaController extends BaseController
{
    private const NIM_LENGTH = 9;

    private Mahasiswa $model;

    public function __construct()
```

```
    {
        $this->model = new Mahasiswa();
    }

    // GET /mahasiswa atau GET /mahasiswa?id=1
    public function index($id = null): void
    {
        if ($id !== null && $id !== '') {
            $id = $this->requireId($id);
            $data = $this->model->find($id);

            if (!$data) {
                Response::json(404, false, 'Data tidak
ditemukan');
            }

            Response::json(200, true, 'Data berhasil
diambil', $data);
        }

        Response::json(
            200,
            true,
            'Data berhasil diambil',
            $this->model->getAll()
        );
    }

    // POST /mahasiswa
    public function store(): void
    {
        $data = $this->clean($this->readJson());

        try {
            $id = $this->model->create($data);
        } catch (DomainException $e) {
            Response::json(409, false,
$e->getMessage());
        }

        Response::json(
            201,
            true,
            'Data berhasil ditambahkan',
            ['id' => (int) $id]
        );
    }
```

```
    // PUT /mahasiswa?id=1
    public function update($id = null): void
    {
        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            Response::json(404, false, 'Data tidak
ditemukan');
        }

        $data = $this->clean($this->readJson());

        try {
            $this->model->update($id, $data);
        } catch (DomainException $e) {
            Response::json(409, false,
$e->getMessage());
        }

        Response::json(200, true, 'Data berhasil
diperbarui');
    }

    // DELETE /mahasiswa?id=1
    public function destroy($id = null): void
    {
        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            Response::json(404, false, 'Data tidak
ditemukan');
        }

        $this->model->delete($id);

        Response::json(200, true, 'Data berhasil
dihapus');
    }

    // Membersihkan dan memvalidasi data
    private function clean(array $input): array
    {
        $data = [
            'nim'   => $this->str($input, 'nim'),
            'nama'  => $this->str($input, 'nama'),
            'email' => $this->str($input, 'email'),
```

```
            'prodi' => $this->str($input, 'prodi'),
        ];

        // Tahap 1: field wajib kosong -> HTTP 400
        $missing = [];

        foreach (['nim', 'nama', 'prodi'] as $field) {
            if ($data[$field] === '') {
                $missing[$field] = "Field $field wajib
diisi";
            }
        }

        if (!empty($missing)) {
            Response::json(
                400,
                false,
                'Field nim, nama, dan prodi wajib
diisi',
                null,
                $missing
            );
        }

        // Tahap 2: validasi format data -> HTTP 422
        $errors = [];

        // NIM hanya boleh berisi angka dan harus tepat
9 digit
        if (!preg_match('/^[0-9]{9}$/D', $data['nim']))
{
            $errors['nim'] =
                'NIM hanya boleh mengandung angka dan
harus tepat 9 digit, tidak boleh mengandung huruf';
        }

        // Nama maksimal 100 karakter
        if ($this->len($data['nama']) > 100) {
            $errors['nama'] = 'Nama maksimal 100
karakter';
        }

        // Prodi maksimal 50 karakter
        if ($this->len($data['prodi']) > 50) {
            $errors['prodi'] = 'Prodi maksimal 50
karakter';
        }
```

```
        // Email opsional, tetapi harus valid jika diisi
        if ($data['email'] !== '') {
            if (
                !filter_var($data['email'],
FILTER_VALIDATE_EMAIL) ||
                $this->len($data['email']) > 100
            ) {
                $errors['email'] =
                    'Format email tidak valid atau
melebihi 100 karakter';
            }
        } else {
            $data['email'] = null;
        }

        // Kirim seluruh kesalahan validasi
        if (!empty($errors)) {
            Response::json(
                422,
                false,
                'Validasi data gagal',
                null,
                $errors
            );
        }

        return $data;
    }
}
```

Pada tahap ini dilakukan peninjauan dan perapian kode pada file `MahasiswaController.php`, khususnya pada method `index()` dan `clean()`. Method `index()` digunakan untuk mengambil seluruh data mahasiswa atau mengambil satu data mahasiswa berdasarkan ID. Hasil pengambilan data dikirimkan dalam format JSON dengan kode status HTTP 200 apabila berhasil dan 404 apabila data tidak ditemukan.

Sementara itu, method `clean()` digunakan untuk memeriksa dan memvalidasi data yang diterima dari permintaan API. Validasi dilakukan terhadap field wajib, format NIM, panjang nama dan program studi, serta format email. Apabila terdapat field wajib yang kosong, sistem memberikan respons HTTP 400, sedangkan

kesalahan format atau aturan validasi menghasilkan respons HTTP 422. Jika data memenuhi ketentuan, data yang telah dibersihkan dikembalikan untuk diproses lebih lanjut.

Melalui penataan tersebut, fungsi pengambilan data dan validasi dapat dipahami dengan lebih jelas. Penggunaan format JSON dan kode status HTTP juga membantu memberikan informasi yang terstruktur kepada klien API ketika melakukan komunikasi dengan server.

C. Langkah 3 — Menambahkan Validasi Data dengan Kode 422

Langkah ketiga adalah menambahkan validasi untuk memastikan data yang diterima API memenuhi ketentuan. Berdasarkan BKPM, format NIM yang tidak sesuai harus menghasilkan HTTP status code `422 Unprocessable Entity`.

```
  private function clean(array $input): array
    {
        $data = [
            'nim'   => $this->str($input, 'nim'),
            'nama'  => $this->str($input, 'nama'),
            'email' => $this->str($input, 'email'),
            'prodi' => $this->str($input, 'prodi'),
        ];

        // Tahap 1: field wajib kosong -> HTTP 400
        $missing = [];

        foreach (['nim', 'nama', 'prodi'] as $field) {
            if ($data[$field] === '') {
                $missing[$field] = "Field $field wajib
diisi";
            }
        }

        if (!empty($missing)) {
            Response::json(
                400,
                false,
                'Field nim, nama, dan prodi wajib
diisi',
                null,
                $missing
            );
```

```
        }

        // Tahap 2: validasi format data -> HTTP 422
        $errors = [];

        // NIM hanya boleh berisi angka dan harus tepat
9 digit
        if (!preg_match('/^[0-9]{9}$/D', $data['nim']))
{
            $errors['nim'] =
                'NIM hanya boleh mengandung angka dan
harus tepat 9 digit, tidak boleh mengandung huruf';
        }

        // Nama maksimal 100 karakter
        if ($this->len($data['nama']) > 100) {
            $errors['nama'] = 'Nama maksimal 100
karakter';
        }

        // Prodi maksimal 50 karakter
        if ($this->len($data['prodi']) > 50) {
            $errors['prodi'] = 'Prodi maksimal 50
karakter';
        }

        // Email opsional, tetapi harus valid jika diisi
        if ($data['email'] !== '') {
            if (
                !filter_var($data['email'],
FILTER_VALIDATE_EMAIL) ||
                $this->len($data['email']) > 100
            ) {
                $errors['email'] =
                    'Format email tidak valid atau
melebihi 100 karakter';
            }
        } else {
            $data['email'] = null;
        }

        // Kirim seluruh kesalahan validasi
        if (!empty($errors)) {
            Response::json(
                422,
                false,
                'Validasi data gagal',
```

```
                null,
                $errors
            );
        }

        return $data;
    }
}
```

Pada langkah ini dilakukan penambahan validasi data pada file `MahasiswaController.php`, khususnya pada method `clean()`. Validasi bertujuan untuk memastikan bahwa data mahasiswa yang dikirimkan melalui API sesuai dengan ketentuan yang telah ditetapkan. Pemeriksaan dilakukan terhadap format dan panjang NIM, panjang nama dan program studi, serta format email.

Apabila data tidak memenuhi aturan validasi, sistem memberikan respons HTTP 422 (Unprocessable Content) dalam format JSON yang berisi pesan kesalahan. Sementara itu, field wajib yang kosong ditangani menggunakan respons HTTP 400 (Bad Request). Dengan adanya validasi tersebut, sistem dapat memberikan informasi kesalahan secara lebih jelas sebelum data diproses lebih lanjut.

D. Langkah 4 — Merapikan Response pada Router

Pada langkah ini dilakukan perapian respons pada file `app/core/Router.php` dengan memanfaatkan class `Response` yang telah dibuat pada langkah sebelumnya. Tujuannya adalah agar respons kesalahan yang dihasilkan oleh Router memiliki format JSON yang konsisten dengan respons pada Controller.

Router memiliki property `$map` yang digunakan untuk menghubungkan nama resource dengan Controller yang menangani permintaan. Pada implementasi ini, resource `mahasiswa` diarahkan ke `MahasiswaController`, sedangkan resource `buku` diarahkan ke `BukuController`.

Ketika resource yang diminta tidak ditemukan dalam pemetaan, Router mengembalikan respons dengan kode HTTP 404 melalui pemanggilan berikut:

```
      if ($resource === null) {
                Response::json(404, false, 'Endpoint
tidak ditemukan');
            }
```

Kode tersebut memastikan bahwa permintaan terhadap endpoint yang tidak terdaftar menghasilkan respons JSON dengan status *Not Found*.

Selanjutnya, Router menangani metode HTTP menggunakan struktur `switch`. Metode GET digunakan untuk mengambil data, POST untuk menambahkan data, PUT dan PATCH untuk memperbarui data, sedangkan DELETE digunakan untuk menghapus data. Apabila metode HTTP yang diterima tidak didukung, Router menjalankan kode berikut:

```
              default:
                    header('Allow: GET, POST, PUT,
PATCH, DELETE');
                    Response::json(405, false, 'Method
tidak diizinkan');
            }
```

Baris `header('Allow: GET, POST, PUT, PATCH, DELETE')` memberikan informasi mengenai metode HTTP yang didukung. Sementara itu, pemanggilan `Response::json()` menghasilkan respons dengan kode HTTP 405 (*Method Not Allowed*).

Selain itu, Router memiliki blok `try-catch` untuk menangani kesalahan yang terjadi selama proses pemrosesan permintaan. Kesalahan dicatat menggunakan `error_log()`, kemudian server mengembalikan respons HTTP 500 dengan pesan umum agar detail kesalahan internal tidak ditampilkan kepada pengguna.

Dengan penerapan tersebut, penanganan kesalahan pada Router menjadi lebih terstruktur. Penggunaan class `Response` juga membantu menjaga konsistensi format JSON sehingga proses pengembangan dan pemeliharaan aplikasi menjadi lebih mudah.

E. Langkah 5 — Pengujian Ulang Seluruh Endpoint

Pada langkah ini dilakukan pengujian ulang terhadap endpoint API mahasiswa menggunakan aplikasi Postman. Pengujian

bertujuan untuk memeriksa fungsi CRUD (*Create, Read, Update, Delete*) serta memastikan bahwa setiap permintaan menghasilkan kode status HTTP dan respons JSON yang sesuai.

Pengujian dilakukan dengan mengakses endpoint mahasiswa melalui server lokal Laragon. Endpoint yang digunakan adalah `http://localhost/BKPM/si-akademik18/public/mahasiswa/`, dengan menyesuaikan alamat URL berdasarkan lokasi proyek yang digunakan.

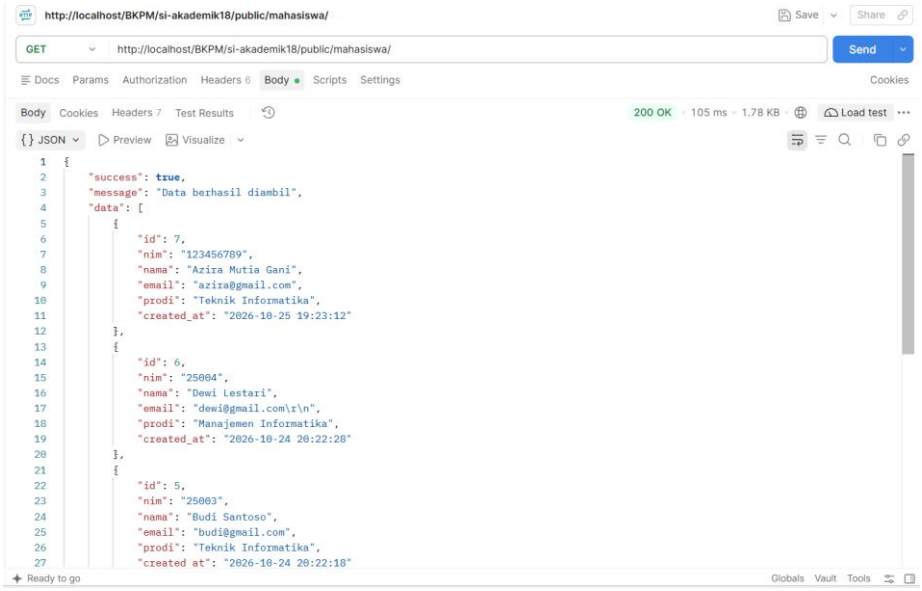
Pengujian meliputi pengambilan data menggunakan metode GET, penambahan data menggunakan POST, perubahan data menggunakan PUT, serta penghapusan data menggunakan DELETE. Selain itu, dilakukan pengujian terhadap kondisi kesalahan, seperti data wajib yang tidak diisi, ID yang tidak ditemukan, dan format NIM yang tidak valid.

Berikut merupakan skenario pengujian yang digunakan:

1. Uji ulang endpoint GET, POST, PUT, dan DELETE pada http://localhost/BKPM/si-akademik18/public/mahasiswa/ menggunakan Postman, pastikan kode status sesuai tabel taksonomi HTTP.

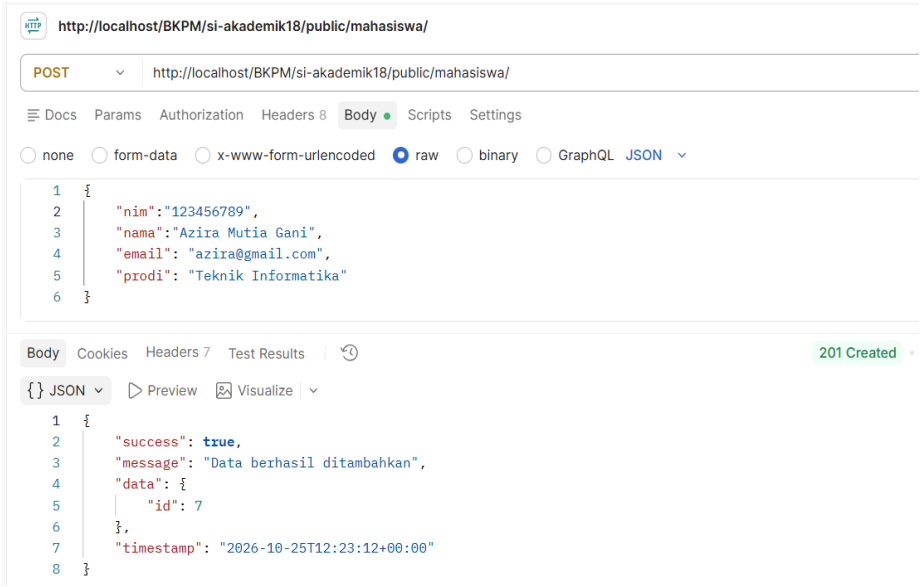
    a) GET Semua mahasiswa

    Pilih GET tanpa parameter id. Harapan: 200 OK.



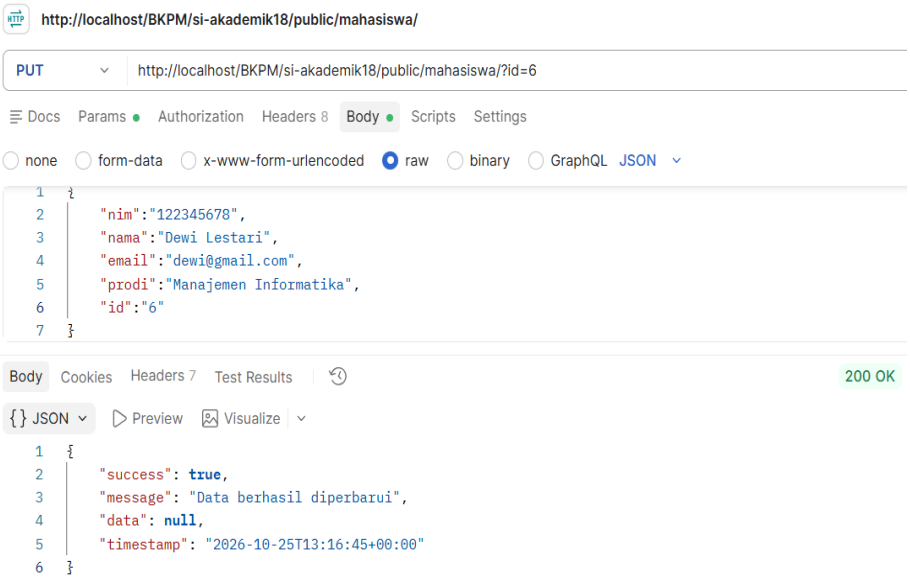
    b) POST tambah mahasiswa

Pilih Body → raw → JSON. Harapan: 201 Created.



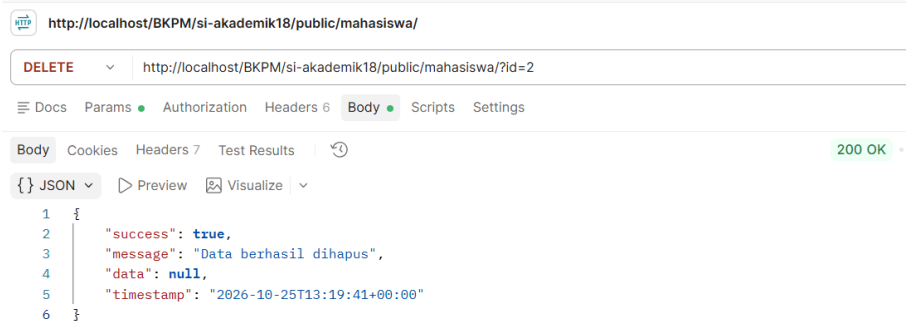
c) PUT ubah mahasiswa

Gunakan ?id=ID_HASIL_POST dan kirim data baru. Harapan: 200 OK.



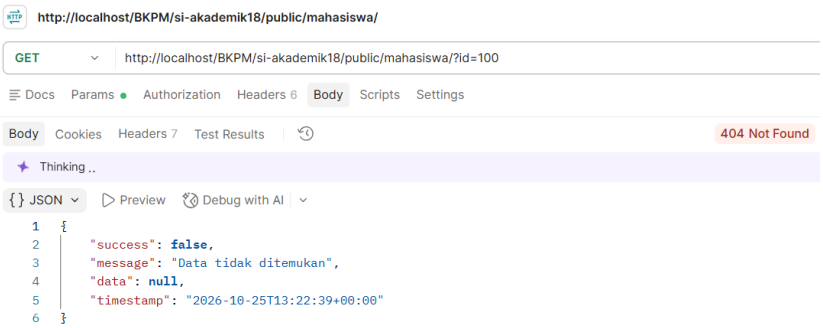
d) DELETE Hapus mahasiswa

Gunakan ?id=ID_HASIL_POST. Harapan: 200 OK.



e) GET ID tidak ditemukan

Gunakan ?id=100. Harapan: 404 Not Found, jika ID tersebut memang tidak ada.



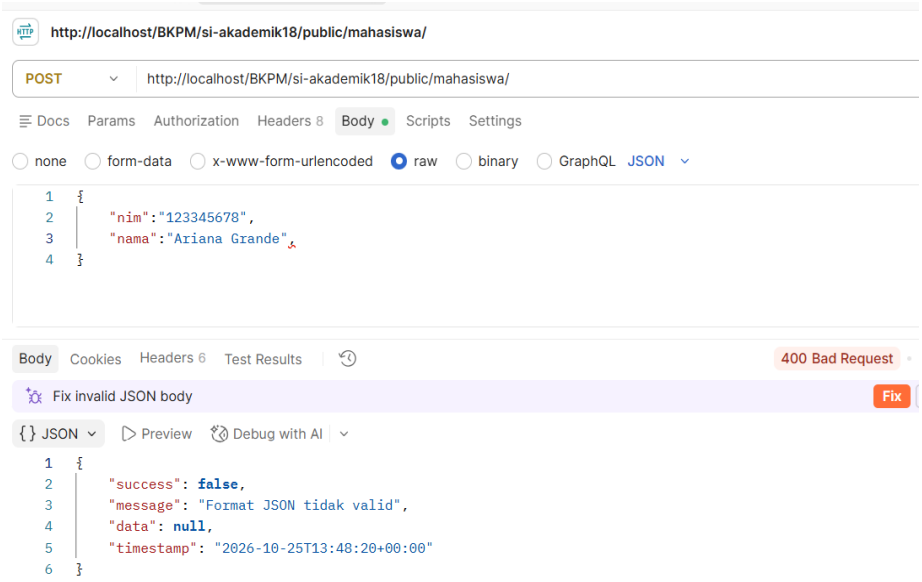
2. Uji skenario gagal (data kosong, id tidak ditemukan, format nim salah) dan pastikan Response::json() mengembalikan kode 400/404/422 beserta pesan yang informatif.

a) JSON tidak valid (400 Bad Request)

http://localhost/BKPM/si-akademik18/public/mahasiswa/

Klik Body → raw → JSON.

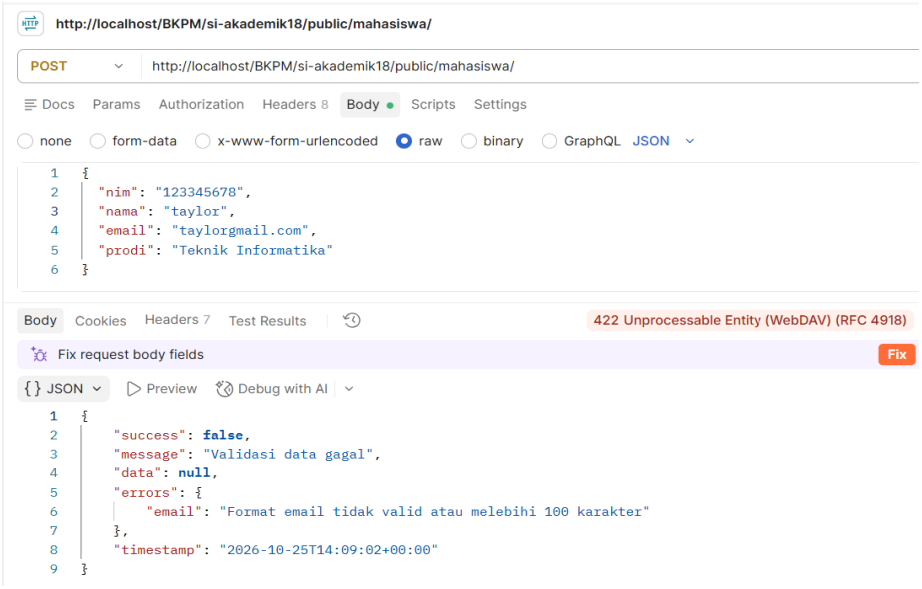
Masukkan JSON yang sengaja salah



b) Data wajib kosong (422)
http://localhost/BKPM/si-akademik18/public/mahasiswa/

Pilih Body → raw → JSON.

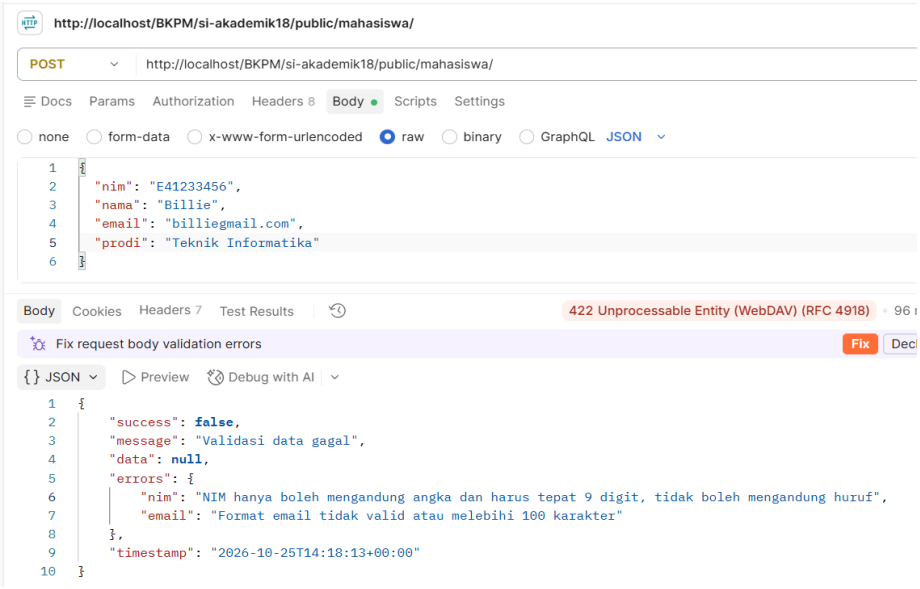
Masukkan data tanpa nama



c) Format NIM salah (422)
http://localhost/BKPM/si-akademik18/public/mahasiswa/

Body → raw → JSON.

Masukkan NIM yang mengandung huruf

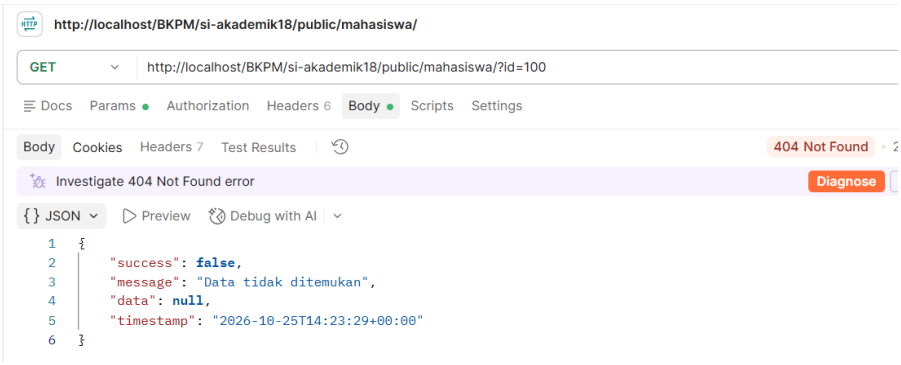


d) ID mahasiswa tidak ditemukan (404)

Pilih metode GET.

http://localhost/BKPM/si-akademik18/public/mahasiswa/?id=100

ID 100 tidak ditemukan di database. Pastikan ID tersebut memang belum digunakan.



Melalui pengujian ini, kesesuaian antara permintaan yang dikirim melalui Postman, kode status HTTP, dan format respons JSON dapat diperiksa. Hasil aktual dari setiap skenario digunakan untuk mengetahui apakah implementasi API telah berjalan sesuai dengan kebutuhan.

3.1.2 Tugas Praktik Mandiri

A. Tugas 1 — Menambahkan Field Errors pada Response::json()

Tugas mandiri pertama adalah menambahkan field `errors` berupa array pada response JSON untuk menampung pesan kesalahan validasi setiap field.

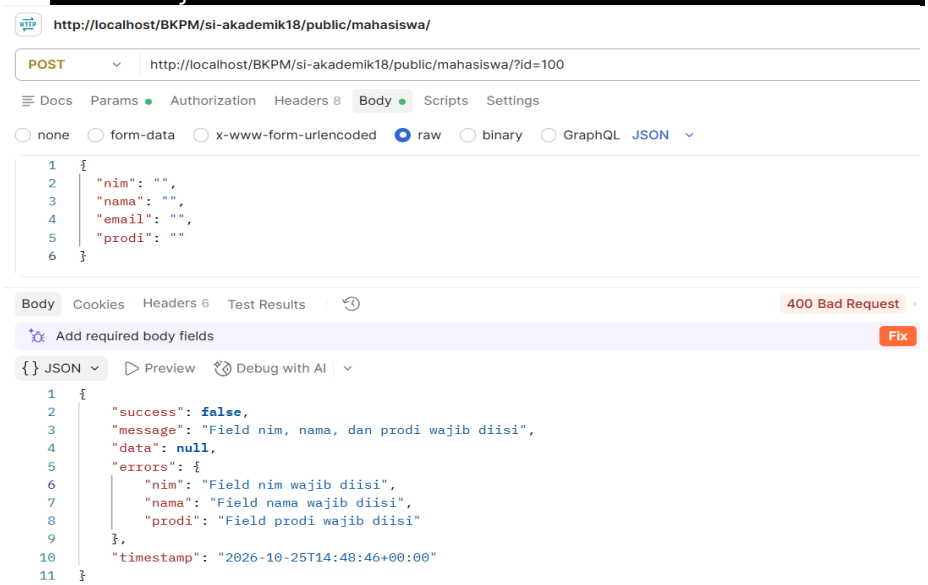
Pada file `app/core/Response.php`, bagian penyusunan response ditambahkan kode berikut:

```php
    if (!empty($errors)) {
            $body['errors'] = $errors;
        }
```

Parameter `$errors` pada method `json()` didefinisikan sebagai array dengan nilai awal kosong. Jika terdapat kesalahan validasi, daftar kesalahan dapat dikirimkan melalui parameter tersebut.

Contoh pemanggilan ketika terjadi kesalahan validasi:

```php
        if (!empty($errors)) {
            Response::json(
                422,
                false,
                'Validasi data gagal',
                null,
                $errors
            );
        }
```



Dengan penerapan ini, client tidak hanya menerima informasi bahwa permintaan gagal, tetapi juga memperoleh keterangan

mengenai field yang bermasalah. Informasi tersebut mempermudah proses pemeriksaan dan perbaikan data yang dikirimkan.

B. Tugas 2 — Menerapkan JSON_PRETTY_PRINT pada Mode Pengembangan

Tugas mandiri kedua adalah menerapkan flag `JSON_PRETTY_PRINT` hanya ketika aplikasi berjalan dalam mode pengembangan. Tujuannya agar response JSON lebih mudah dibaca selama proses pengembangan tanpa harus selalu menggunakan format berindentasi pada lingkungan produksi.

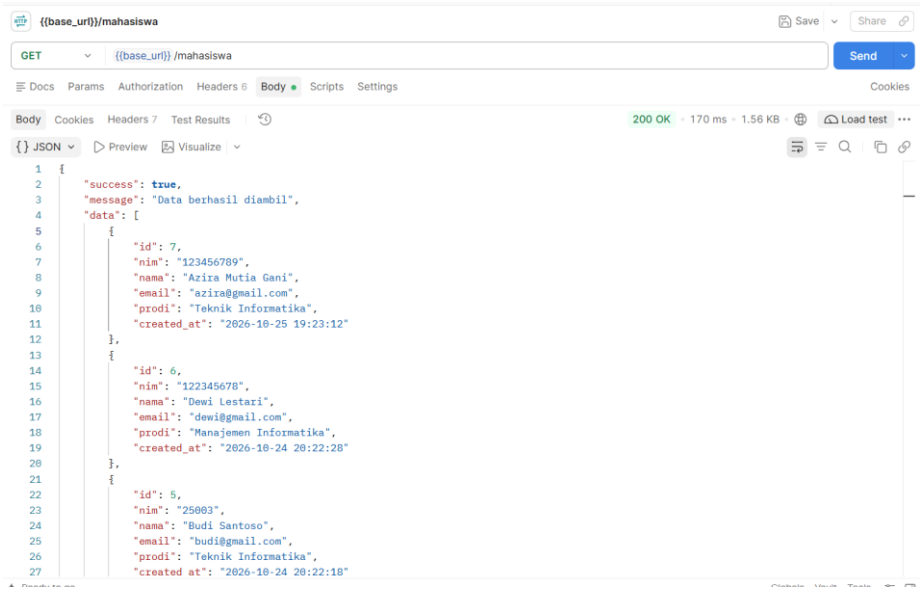
Pada proyek ini, konstanta `APP_ENV` didefinisikan pada file `app/config/app.php`. Pengaturan flag dilakukan pada class `Response`, seperti berikut:

```
     if (self::isDevelopment()) {
            $flags |= JSON_PRETTY_PRINT;
        }

        echo json_encode($body, $flags);
        exit;
    }

    private static function isDevelopment(): bool {
        return defined('APP_ENV') && APP_ENV ===
'development';
    }
}
```

Ketika `APP_ENV` bernilai `development`, flag `JSON_PRETTY_PRINT` ditambahkan sehingga response JSON ditampilkan dengan format yang lebih rapi. Apabila aplikasi tidak berada dalam mode pengembangan, flag tersebut tidak ditambahkan.



3.1.3 Analisis Hasil Praktikum

Pada praktikum ini dilakukan pengembangan class `Response` untuk meningkatkan pengelolaan response API dalam format JSON. Pengembangan meliputi penambahan field `errors` untuk menampilkan informasi kesalahan validasi dan penerapan `JSON_PRETTY_PRINT` pada mode pengembangan.

Pada tugas pertama, field `errors` digunakan untuk memberikan informasi mengenai kesalahan pada input pengguna. Ketika terdapat field wajib yang belum diisi, sistem dapat mengembalikan response dengan status HTTP 400. Sementara itu, kesalahan format atau validasi data, seperti NIM yang mengandung huruf, ditangani menggunakan status HTTP 422. Informasi kesalahan ditampilkan dalam bentuk objek `errors`, sehingga penyebab kegagalan permintaan dapat diketahui dengan lebih jelas.

Pada tugas kedua, penggunaan `JSON_PRETTY_PRINT` diatur melalui method `isDevelopment()`. Method tersebut memeriksa nilai konstanta `APP_ENV` untuk menentukan apakah aplikasi berjalan dalam mode pengembangan. Jika nilainya adalah `development`, response JSON ditampilkan dengan indentasi agar lebih mudah dibaca dan diperiksa melalui Postman. Jika kondisi tersebut tidak terpenuhi, format pretty print tidak diterapkan.

Berdasarkan implementasi tersebut, pengembangan class `Response` membantu membuat response API lebih informatif dan memudahkan proses pemeriksaan kesalahan. Penambahan informasi validasi membantu pengguna mengetahui penyebab kesalahan input, sedangkan pengaturan format JSON mempermudah proses debugging selama pengembangan aplikasi.

BAB 4. PEMBAHASAN

4.1 Pembahasan

Praktikum Acara 18 membahas penerapan JSON encoding dan HTTP status code pada RESTful API Sistem Informasi Akademik menggunakan PHP Native. Pengembangan dilakukan dengan menambahkan class `Response.php` pada lapisan Core untuk menyeragamkan format respons yang dikirimkan server kepada client.

Class `Response` memiliki method `json()` yang bertanggung jawab mengatur kode status HTTP, header respons, struktur data JSON, serta timestamp. Data yang akan dikirimkan disusun dalam bentuk array, kemudian diubah menjadi format JSON menggunakan fungsi `json_encode()`. Penggunaan class tersebut membuat proses pembentukan respons lebih terstruktur dan mengurangi penulisan kode yang berulang pada Controller maupun Router.

Penerapan HTTP status code disesuaikan dengan kondisi pemrosesan request. Status `200 OK` digunakan ketika permintaan berhasil diproses, sedangkan `201 Created` digunakan ketika data baru berhasil ditambahkan. Status `400 Bad Request` digunakan ketika terdapat field wajib yang belum diisi, sementara `404 Not Found` digunakan ketika data atau endpoint tidak ditemukan. Selain itu, status `405 Method Not Allowed` digunakan ketika metode HTTP yang diminta tidak diizinkan, `409 Conflict` digunakan ketika terjadi konflik data, dan `422 Unprocessable Content` digunakan ketika data tidak memenuhi aturan validasi.

Pada `MahasiswaController.php`, method `clean()` digunakan untuk membersihkan dan memvalidasi data mahasiswa sebelum diproses lebih lanjut. Validasi mencakup pemeriksaan field wajib, format NIM yang harus terdiri dari sembilan digit angka, panjang nama dan program studi, serta format email apabila diisi. Jika ditemukan kesalahan, sistem mengirimkan respons JSON yang berisi pesan kesalahan dan kode status yang sesuai.

Selain itu, Router diperbarui agar kesalahan seperti endpoint yang tidak ditemukan dan metode HTTP yang tidak diizinkan dapat dikembalikan menggunakan class `Response`. Dengan demikian, format respons kesalahan pada Router dan Controller dapat dibuat lebih konsisten.

Praktikum juga mencakup tugas mandiri berupa penambahan field `errors` dan penerapan `JSON_PRETTY_PRINT` pada mode pengembangan. Field `errors` digunakan untuk menyampaikan informasi kesalahan pada setiap field yang tidak memenuhi ketentuan. Sementara itu, `JSON_PRETTY_PRINT` digunakan agar struktur JSON lebih mudah dibaca ketika aplikasi berjalan dalam mode pengembangan. Kedua fitur tersebut membantu proses pemeriksaan respons API dan mempermudah proses debugging.

4.2 Analisis Pengujian

Pengujian RESTful API dilakukan menggunakan Postman dengan mengirimkan request ke endpoint mahasiswa pada server lokal Laragon. Pengujian mencakup pemeriksaan respons berhasil, respons gagal, validasi data, serta format JSON yang dihasilkan oleh class `Response`.

Pada pengujian pengambilan data menggunakan metode GET, respons yang diharapkan adalah HTTP `200 OK` apabila permintaan berhasil diproses. Pada penambahan data menggunakan POST, respons yang diharapkan adalah HTTP `201 Created` apabila data berhasil ditambahkan. Metode PUT digunakan untuk memperbarui data dengan respons HTTP `200 OK` apabila proses berhasil, sedangkan metode DELETE digunakan untuk menghapus data dengan respons sesuai implementasi aplikasi.

Pengujian juga mencakup kondisi kesalahan. Ketika field wajib dikosongkan, sistem diharapkan mengembalikan HTTP `400 Bad Request` disertai informasi field yang belum diisi. Ketika NIM tidak memenuhi aturan validasi, seperti mengandung huruf atau tidak berjumlah sembilan digit, sistem diharapkan memberikan HTTP `422 Unprocessable Content` beserta keterangan kesalahan pada field `nim`. Sementara itu, permintaan terhadap data yang tidak ditemukan diharapkan menghasilkan HTTP `404 Not Found`.

Pada pengujian tugas mandiri pertama, keberadaan field `errors` diperiksa untuk memastikan bahwa respons kesalahan menyertakan informasi validasi yang relevan. Field tersebut hanya ditambahkan apabila terdapat kesalahan, sehingga respons yang tidak memiliki kesalahan validasi tidak perlu menampilkan objek `errors`.

Pada pengujian tugas mandiri kedua, format respons JSON diperiksa ketika konstanta `APP_ENV` bernilai `development`. Dalam kondisi tersebut, respons diharapkan menggunakan indentasi melalui `JSON_PRETTY_PRINT`. Apabila aplikasi tidak berada dalam mode pengembangan, flag tersebut tidak ditambahkan.

Hasil pengujian perlu dinilai berdasarkan status HTTP dan isi respons yang benar-benar ditampilkan pada Postman. Kesesuaian hasil aktual dengan hasil yang diharapkan menunjukkan apakah implementasi telah berjalan sesuai ketentuan. Jika terdapat perbedaan, pemeriksaan dapat dilakukan pada class `Response`, validasi di Controller, konfigurasi lingkungan aplikasi, serta penanganan request pada Router.

BAB 5. PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum Acara 18, dapat disimpulkan bahwa JSON encoding dan HTTP status code merupakan bagian penting dalam pengembangan RESTful API. JSON encoding menggunakan fungsi `json_encode()` untuk mengubah data PHP menjadi format JSON, sedangkan HTTP status code digunakan untuk menunjukkan hasil pemrosesan request oleh server.

Pembuatan class `Response.php` membantu menyeragamkan struktur respons API yang terdiri atas indikator keberhasilan, pesan, data, dan timestamp. Penambahan field `errors` juga memungkinkan sistem memberikan informasi kesalahan validasi secara lebih terperinci. Selain itu, penerapan `JSON_PRETTY_PRINT` pada mode pengembangan membantu menampilkan respons JSON dengan format yang lebih mudah dibaca.

Penerapan class `Response` pada Controller dan Router mendukung pengelolaan respons yang lebih terstruktur. Pengujian menggunakan Postman dapat membantu memeriksa kesesuaian metode HTTP, kode status, dan isi respons dengan kondisi permintaan. Dengan demikian, praktikum ini memberikan pemahaman mengenai pengelolaan respons dan penanganan kesalahan dalam pengembangan RESTful API menggunakan PHP Native.

5.2 Saran

Pengembangan RESTful API selanjutnya sebaiknya tetap memperhatikan konsistensi format respons JSON dan penggunaan HTTP status code sesuai kondisi permintaan. Validasi data perlu diterapkan secara tepat agar data yang masuk ke basis data memenuhi ketentuan yang telah ditetapkan.

Selain itu, pengujian menggunakan Postman sebaiknya mencakup kondisi berhasil dan gagal, termasuk field wajib yang kosong, format data yang tidak valid, endpoint yang tidak ditemukan, serta metode HTTP yang tidak diizinkan. Pengaturan lingkungan aplikasi juga perlu diperiksa agar fitur seperti `JSON_PRETTY_PRINT` berjalan sesuai konfigurasi.

Setiap perubahan pada Controller, Router, dan class `Response` sebaiknya diuji kembali untuk memastikan tidak menimbulkan kesalahan pada fungsi yang

sudah tersedia. Dokumentasi endpoint dan hasil pengujian juga perlu disusun secara sistematis agar proses pemeliharaan dan pengembangan aplikasi pada praktikum berikutnya menjadi lebih mudah.